

AI Engineering Boot Camp

Day 8 — Pandas & Data Engineering

DataFrames · Cleaning · EDA · Save & Load

Target: Google · Meta · OpenAI · Anthropic

Files Created Today

- `day8_pandas_basics.py` — DataFrame creation, exploring, selecting, filtering
- `day8_pandas_cleaning.py` — missing values, duplicates, filling, groupby, sort
- `day8_eda.py` — full exploratory data analysis, feature engineering, correlation
- `day8_save_load.py` — saving and loading CSV and JSON

All files pass `mypy --strict`. All pushed to GitHub.

Why Pandas?

NumPy is great for math on arrays of the same type. But real data is messy — mixed types, missing values, labeled columns. Pandas handles all of this. Under the hood it uses NumPy for math, but adds the structure and tools for real-world data.

Every dataset you load — CSV files, database results, API responses — becomes a DataFrame. You spend most of your time in ML cleaning, filtering, and transforming DataFrames before training anything.

Core Data Structures

Series — 1D labeled array (one column)

```
s = pd.Series([85, 92, 78], name='score')
# 0      85
# 1      92
# 2      78
```

DataFrame — 2D table with labeled rows and columns

```
df = pd.DataFrame({
    'name':    ['Alice', 'Bob', 'Charlie'],
    'score':   [88.5, 72.0, 95.5],
    'passed':  [True, False, True]
})
```

Every dataset becomes a DataFrame. df is the universal convention — you'll see it in every ML codebase, every Kaggle notebook, every data science tutorial worldwide.

The 5 Things You Do With Every Dataset

Step	What	Key methods
1. Load	Read data into a DataFrame	pd.read_csv(), pd.read_json(), pd.DataFrame()
2. Explore	Understand shape, types, gaps	df.shape, df.head(), df.info(), df.describe()
3. Clean	Fix missing, wrong types, dupes	dropna(), fillna(), drop_duplicates(), astype()
4. Transform	Filter, create columns, aggregated	df[condition], groupby(), sort_values(), apply()
5. Export	Save back or pass to model	to_csv(), to_json(), df.values (NumPy array)

Exploring Data

```
df.shape          # (rows, columns) – always check first
df.head()         # first 5 rows
df.tail()         # last 5 rows
df.info()         # column names, types, non-null counts
df.describe()     # stats: mean, std, min, max per column
df.columns        # column names as Index
df.dtypes         # data type per column
df.isnull().sum() # count missing values per column
df['col'].value_counts() # frequency of each value
```

Selecting Data

Syntax	What it returns
df['name']	One column — Series
df[['name', 'score']]	Multiple columns — DataFrame
df.iloc[0]	Row by position number
df.loc[0]	Row by label/index

<code>df[df['score'] > 80]</code>	Filter rows where condition is True
<code>df[(df['score'] > 80) & (df['passed'] == True)]</code>	Multiple conditions — use & not 'and'

Critical: use & for AND, | for OR in Pandas conditions. Python's 'and'/'or' keywords don't work on Series.

Cleaning Data

Missing values

```
df.isnull()           # True/False where missing
df.isnull().sum()     # count missing per column
df.dropna()           # drop rows with any missing
df.fillna(0)           # fill all missing with 0
df['age'].fillna(df['age'].median()) # fill with median
df['name'].fillna('unknown') # fill with string
```

Why median over mean for filling missing values?

Median is robust to outliers. If ages are [24, 25, 22, 27, 95] — the 95 is likely an error. Mean = 38.6 (pulled up by outlier). Median = 25 (not affected). Filling with a bad value corrupts training data.

Duplicates and types

```
df.drop_duplicates()  # remove duplicate rows
df['score'].astype(float) # convert column type
df.reset_index(drop=True) # reset row numbers after filtering
```

Always use .reset_index(drop=True) before saving a filtered DataFrame. Without it, the original index is saved as a phantom extra column in the CSV.

Transforming Data

Creating new columns

```
df['passed'] = df['score'] > 75
```

```
df['grade'] = df['score'].apply(lambda x: 'A' if x >= 90 else 'B')
df['income_per_exp'] = df['income'] / (df['experience_years'] + 1)
```

Groupby and aggregate

```
df.groupby('grade')['score'].mean() # avg score per grade
df.groupby('hired')['income'].mean() # avg income by hired status
```

Sorting and renaming

```
df.sort_values('score', ascending=False) # sort descending
df.rename(columns={'score': 'test_score'}) # rename column
```

Correlation

```
df.corr(numeric_only=True).round(2) # correlation matrix
```

Value	Meaning	Example
1.0	Perfect positive — one up, other up	temperature vs ice cream sales
0.0	No relationship — useless feature	random noise vs target
-1.0	Perfect negative — one up, other down	temperature vs hot drink sales
0.5 to 0.8	Moderate positive correlation	education vs income

Before training any model: check correlations. Features near 0.0 with the target have no predictive power — drop them. Features highly correlated with each other are redundant — keep one.

Saving and Loading Data

CSV — most common format

```
df.to_csv('data.csv', index=False) # save — always use index=False
df = pd.read_csv('data.csv') # load
```

JSON

```
df.to_json('data.json', orient='records', indent=2) # save
df = pd.read_json('data.json') # load
```

Always use index=False when saving to CSV. Without it, Pandas saves the row numbers as an extra column called 'Unnamed: 0' that corrupts downstream code.

Actual Outputs From Today's Code

day8_pandas_basics.py

```
shape: (5, 4)
name column: 0    Alice  1    Bob  2    Charlie  ...
passed students: Alice(88.5) Charlie(95.5) Eve(83.5)
high scorers: Alice Charlie Eve
```

day8_pandas_cleaning.py

```
missing values: name 1 age 1 score 1 grade 0
after fillna: unknown, median age, mean score filled
after drop_duplicates: Bob row 5 removed
average score by grade: A=92.0 B=74.9 C=61.0
```

day8_eda.py

```
shape: (200, 5)
hired distribution: True=102 False=98
avg income hired vs not: False=52938 True=51207
all correlations near 0.0 – random data has no signal
senior hired: 56 out of 112 seniors
```

day8_save_load.py

```
saved to data.csv loaded shape: (100, 4)
saved to data.json loaded shape: (100, 4)
saved 49 hired people to hired_only.csv
verified shape: (49, 4) all hired: True
```

Anki Cards — Day 8

- Q: What does `df.describe()` show? A: Stats — count, mean, std, min, max for each column
- Q: What does `df.isnull().sum()` do? A: Counts missing values per column
- Q: Why use median over mean to fill missing values? A: Median is robust to outliers — not pulled by extreme values
- Q: What does correlation of 0.0 mean? A: No relationship — feature has no predictive power
- Q: What does `.reset_index(drop=True)` do? A: Resets row numbers after filtering — prevents phantom index column
- Q: What operator do you use for AND conditions in Pandas? A: `&` not 'and' — `df[(cond1) & (cond2)]`
- Q: What does `df.groupby('col')['val'].mean()` do? A: Groups by col and calculates mean of val for each group
- Q: Why use `index=False` when saving CSV? A: Prevents Pandas saving row numbers as extra 'Unnamed: 0' column
- Q: What is a DataFrame? A: A 2D table with labeled rows and columns — the core Pandas data structure
- Q: What is a Series? A: A 1D labeled array — a single column of a DataFrame

Next: [Day 9 — Matplotlib visualisation](#), then [Day 10 — your first real ML model with scikit-learn](#).